7.6.1 Log requests on the server

If you're in charge of the server that receives the request, you could configure it to log HTTP headers. The following code will log the request headers to a Node.js server's output:

```
console.log(req.headers);
```

The headers can be logged to the output, to the server logs, or to anywhere that is convenient for you. The exact configuration of where to log the headers depends on your server and logging framework.

7.6.2 Use test-cors.org

We covered test-cors.org in the previous section. Because test-cors.org is a web tool, it can be accessed from any browser, including mobile browsers. If you visit test-cors.org from your mobile phone, you can send a series of test requests to your own server or to the test-cors.org server and view the response.

7.6.3 Use remote debugging tools

There are remote debugging tools available for mobile phones and they all work in the same way. You connect your phone to your computer, and the computer logs request and response information from the phone. Here are a few remote debugging options:

- Chrome has built-in remote debugging options for Android phones. More details can be found at https://developer.chrome.com/devtools/docs/remote-debugging.
- Safari supports remote debugging for iPhones and iOS devices, but only from a Mac. More details can be found at http://webdesign.tutsplus.com/articles/quick-tip-using-web-inspector-to-debug-mobile-safari–webdesign-8787.
- Fiddler is a tool that allows you to proxy mobile requests through your computer, and logs the request and response details. Fiddler can be used with both iOS and Android, but it only runs on Windows. You can learn more about Fiddler at www.telerik.com/fiddler.

7.6.4 Use a mobile simulator

Google, Apple, and Microsoft all provide mobile simulators to test their respective mobile platforms. These simulators are programs that run in your computer, but mimic a mobile phone interface. Figure 7.20 shows an example of Apple's iOS simulator running on a Mac.

Using a simulator will give you a good idea of how a request will behave on a mobile device. It's also handy when you want to test a device you may not have access to (for example, if you own an Android phone, but want to test an iPhone).

So far I've covered techniques for testing and debugging CORS requests. But what if you've tried all these techniques and you're still stuck? The next section looks into some options for getting help with CORS questions.